Agentic RL on Terminal-Bench with slime and AgentCore Runtime

One task, one image, one Runtime; a fresh microVM per episode, pre-opened and heartbeated

Kaige Yang
Senior Applied Scientist, AWS

September 30, 2026 · `code/demos/demo_cpu_terminalbench`

Key conclusions: the sandbox is solved, the reward is the bottleneck

Table 1 **The setting**

Use case	RL-train Qwen3-4B (slime GRPO) on tasks that each need **their own OS image** (slide 4)
Isolation	One episode = one new session = one dedicated microVM, sanitized at the end
Data	241 Terminal-Bench tasks → 218 images and Runtimes → 165 rows (slides 5–6)

Table 2 **Four conclusions and the evidence behind each**

	Conclusion	Evidence	Slides
1	**One image per row works** on AgentCore	218 of 229 arm64 builds ship (p50 127 MB, slide 7); 382 episodes on 382 fresh microVMs, **0 infra errors**	slide 15
2	**Cold start is the cost; pre-open removes it**	509 of 868 concurrent opens boot cold (6–49 s); with pre-open, boot wait **p95 = 0**, 374 of 382 never waited	slides 10–12
3	**Heartbeat is required**: idle reclaim is silent	17 of 382 pre-opened sessions idled past 15 min; pings **kept all 17**, 0 failed	slides 13–14
4	**The reward, not the sandbox, blocks learning**	4 of 320 solved; 38 of 40 GRPO groups had no gradient; test-fraction reward gives **2 → 17**	slides 17–18

In plain English: one image per row costs a cold boot and a silent 15-minute reclaim; pre-open and heartbeat remove both, so the next run is about the reward.

Source: Smoke run 2026-09-23, warm-start and cold-start probes 2026-09-25/29, `terminalbench/tasks/images.csv`; checked 2026-09-30.

© 2026, Amazon Web Services, Inc. or its affiliates.

The demo: use case covered, why this demo, design considerations

Table 3 The demo in three answers

Use case covered	Agentic RL where **every task needs a different OS image** (packages, services, files, compilers) and the agent changes the whole machine through the shell: hundreds of bespoke images, each behind its own AgentCore Runtime, with strict row → image → Runtime routing.
Why this demo	The hardest sandbox shape in the literature datasets: no generic image can serve it, and a shared session would leak one episode's changes into the next. It shows AgentCore scaling to a **pool of 218 Runtimes** built from the tasks' own Dockerfiles with no per-task code, and the two latency fixes a per-episode microVM needs: **pre-open** and **heartbeat**.
Design considerations	**One fresh microVM session per episode** (`terminalbench_pool.py`), stopped once graded · **look-ahead pre-open** of the next 2 batches (`TB_PREOPEN_BATCHES=2`) · **heartbeat** on by default · the **mini_claude loop runs inside the session** and dials the trainer's `AnthropicAdapter` back over the VPC (SG→SG, 18100–18163) · **harness tier** (`HarnessPool`, W = RB × NS, least-loaded engine, sticky per episode) · each task image is **wrapped, not rewritten** (standalone `/opt/acs/py` + stdlib server layer) · reward = the task's **hidden pytest oracle** (RLVR, no judge) · of 229 tasks, **7 fail to build** for arm64 and **4 exceed the 2 GB cap**.

Table 4 Status, 2026-09-30

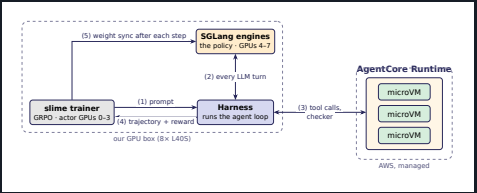
Scope	`code/demos/demo_cpu_terminalbench` is the only demo since 2026-09-30. The long run is `convergence/20_cpu_terminalbench.sh`.
Smoke run	2026-09-30 re-run on the current code (`5_run.sh`): **10 steps + final eval in 22 min**, 449 episodes, **0 infra errors**, 22/387 train and 1/62 eval solved (slide 16). The deep-dive slides use the 2026-09-23 smoke.

Source: `code/demos/demo_cpu_terminalbench/README.md`; checked 2026-09-30.

© 2026, Amazon Web Services, Inc. or its affiliates.

The sandbox is the reward source; to AgentCore an RL batch is just Runtimes and session ids

Figure 1 Agentic RL: trainer, inference, harness, sandbox



- **slime** trains, **SGLang** serves, the **harness** records tokens + logprobs.
- **AgentCore** runs tool calls and hidden tests: **it produces the reward**.
- **No slime fork:** three plugin hooks (generate, reward, data source).

Table 5 AgentCore concepts seen from agentic RL

AgentCore	RL meaning	Terminal-Bench, one step
Image	the environment (ECR, arm64, $\leq$ 2 GB)	one per task, 218 deployed
Runtime	a task's env behind one ARN, reusable	4 tasks drawn $\longrightarrow$ 4 Runtimes
Session	one rollout; its first invoke boots it	4 $\times$ 8 = **32 new sessions**
microVM	isolation of that rollout (2 vCPU, 8 GB)	32 VMs, stopped at episode end
GRPO group	8 sessions on **one** Runtime	same start state, no sharing

Table 6 Lifecycle facts that shape the RL design

No "create session" call	a new id's first invoke is a **cold start** $\longrightarrow$ pre-open (slide 10)
15-min idle reclaim, silent	a waiting session comes back empty $\longrightarrow$ heartbeat (slide 13)
5000 sessions · 25 new/s · 1000 Runtimes	bounds rollouts in flight, burst rate, distinct images [Q1][Q2]

Source: `demo_cpu_terminalbench/agentcore_slime_demo/generate.py`, `harness_pool.py`; `demo_cpu_terminalbench/run.sh` (RB = 4, NS = 8); AgentCore docs [D2], quotas [Q1][Q2] (slide 19); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

A task is a whole machine in a known state, so every row needs its own image

Table 7 **The files of one task (`acl-permissions-inheritance`)**

`task.yaml`	Instruction (the prompt), difficulty, test timeout
`Dockerfile`	**The environment**: base image, packages, users, files
`solution.sh`	Reference solution (oracle), never shown to the agent
`tests/`, `run-tests.sh`	Hidden pytest checks, injected only at grading

Example: *"Make `/srv/shared` setgid with default ACLs for `alice` and `bob`."* The tests write as each user and check access. **Reward = 1 iff every hidden test passes.**

Table 8 **What the 226 Dockerfiles add (3 tasks use a prebuilt image)**

Install packages (`apt`, `pip`, `npm`, ...)	179
Copy task files (source, data, broken configs)	142
Download at build time (`git clone`, models)	68
Nothing beyond the base image	14

Why not one shared image?

- **States contradict:** `broken-python` deletes `pip`; others need it.
- **Size:** 48 GB in total vs a 2 GB cap.
- **Reward validity:** tests read machine state.

Why not install at episode start?

- Builds take p50 6.8 min and GRPO siblings could diverge.

In plain English: the task is not a question, it's a computer in a known state plus a list of checks.

Source: `laude-institute/terminal-bench` (main), `terminalbench/tasks/*/Dockerfile`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

241 tasks → 218 images → 165 rows: the oracle check keeps only trustworthy rewards

Table 9 Where the 75 tasks go

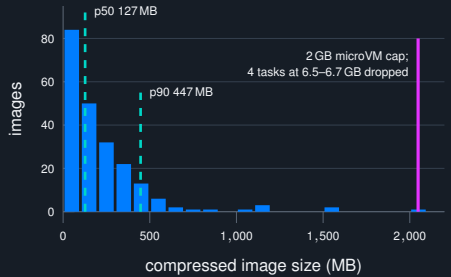
Stage	Left	Drop	Why
Terminal-Bench	241		all tasks
Single container	229	12	need several containers (`postgres`, `redis`, ...); one session = one VM
Built, ≤ 2 GB	218	11	7 fail on arm64, 4 over the cap (slide 7)
Oracle-usable	166	52	fail the oracle check (below)
Train · eval	134 · 31	1	`fix-git`: latest rebuild failed (`git clone`); disjoint tasks, every 5th per difficulty to eval

Table 10 The oracle check: two runs, each on a fresh session

Run	Does	Tests must	Drop
Null	leave the machine untouched, run hidden tests	✗ fail	0
Oracle	run `solution.sh`, then hidden tests	✓ pass	52

- **Why:** otherwise reward is 1 for doing nothing, or 0 whatever the policy does. It runs on AgentCore because solutions target x86 Docker, not arm64 microVMs.
- **The 52:** solution passes no test 27, some 18, none collected 2; invoke errors 5. Only 3 causes logged (x86-only wheel, `rustc` drift, no network); **rest not yet investigated**.
- **218 images, 165 rows:** the check needs a deployed Runtime, so the 52 are deployed but never routed to.

Figure 2 Size of the 218 built images (100 MB bins)



- **Total 48 GB**, max 2.0 GB; 201 of 218 under 500 MB.
- 7 over 1 GB (models, QEMU disks); size sets the cold boot (slide 10).

Source: `terminalbench/tasks/{manifest.json, images.csv` (incl. 54 MB layer), `oracle.csv}` from `validate_oracle.py`; `data/terminalbench_*.jsonl`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Of 229 arm64 builds, 218 ship; 4 exceed 2 GB and 7 fail (4 x86-only, 3 downloads)

Table 11 arm64 build of the 229 single-container tasks

Outcome	Tasks	Detail
✔ Shipped, ≤ 2 GB	218	61 MB–2.0 GB, p50 127 MB, 48 GB (slide 6)
✘ Over 2 GB	4	`mteb-eval`, `-leaderboard`, `-retrieve` 6.7 GB; `png-generation` 6.5 GB: 3 × the cap
✘ *Build failed: 7*		
x86-only	4	`qemu-alpine-ssh`, `qemu-startup`: no `qemu-system-x86` on arm64 · `reverse-engineering`: x86 assembly · `accelerate-maximal-square`: likely x86-only base
Download failed	3	`leelachess0-pytorch-conversion` (lczero), `word2vec-from-scratch` (HF), `fix-git` (`git clone`; `slurm-simple-node-monitoring` built on retry): a retry may fix

- **Build time** p50 6.8 min, max 2.0 h; **39.6 h** total, 16 in parallel.
- **arm64** (microVMs' only arch), cross-built under QEMU.
- **One ECR repo + one Runtime per task**; resumable.

`Dockerfile.acs` (generated per task)

```
# ... the task's own Dockerfile, unchanged ...
FROM ghcr.io/laude-institute/t-bench/ubuntu-24-04:latest
RUN apt-get update && apt-get install -y acl util-linux
# ===== sandbox layer (appended by deploy_terminalbench.py)
USER root
COPY --from=acs py /opt/acs/py # standalone Python
COPY --from=acs acs_http_server.py sandbox_server_agent.py /opt/acs/
RUN /opt/acs/py/bin/python3 -c "import http.server, tarfile, json,
        subprocess"
EXPOSE 8080
CMD ["/opt/acs/py/bin/python3", "/opt/acs/acs_http_server.py"]
```

- **Own Python**: never touch the task's (it is graded).
- **Stdlib HTTP server** on 8080: `exec`, file put/get.
- **No tests in the image**: injected after the agent stops.

Source: `terminalbench/tasks/images.csv` (`status`, `size_mb`, `error`, `build_s`), build 2026-09-23; `deploy_terminalbench.py`, `acs_ctx/`; Runtime quota [Q1] (slide 19); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Four scripts build the pool once; each run is then oracle check, data, train, clean up

From `code/demos/demo_cpu_terminalbench/`, on the host

```
# --- build the pool (once) ---
TB_LIMIT=2 bash 2_build_image.sh # 1+2. fetch, dry run
TB_LIMIT=2 bash 4_deploy_runtime.sh
bash 2_build_image.sh # 3. all tasks, 16 parallel
bash 3_push_ecr.sh # report: images.csv
bash 4_deploy_runtime.sh # 4. -> arn_by_image.json

# --- every run ---
docker exec agentcore_p4 python3 \
  /work/code/demos/demo_cpu_terminalbench/terminalbench/validate_oracle.py # 5.
bash 1_prepare_data.sh # 6. usable rows only
bash 5_run.sh # 7. 10 steps + 1 eval
bash 6_cleanup.sh # 8. delete 218 Runtimes
```

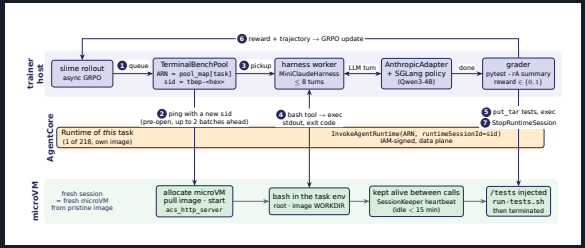
1. Fetches the corpus once; keeps the 229 single-container tasks.
2. Catches layer and IAM errors in minutes.
3. Failed and over-cap builds are skipped.
4. VPC Runtimes, NAT egress, SG→SG to the adapter.
5. Oracle check (slide 6).
6. 165 usable rows → 134 train, 31 eval.
7. In `agentcore_p4`; 8 GPUs (4 train + 4 infer).
8. ECR images are kept.

Source: `demo_cpu_terminalbench/README.md`, `deploy/demo.env`, `5_run.sh`, `run.sh` (standalone demo again since 2026-09-30; no `VARIANT=`/`DATASET=` needed); checked 2026-09-30.

© 2026, Amazon Web Services, Inc. or its affiliates.

One episode: route, fresh session, agent loop in the microVM, grade, stop

Figure 3 route → fresh session → agent loop → grade → stop



1. **Route and open:** `task.image` → Runtime ARN; take a pre-opened session. A new id's first invoke **is** the boot.
2. **Agent loop in the microVM**, calling the policy over a VPC dial-back; up to 8 turns.
3. **Grade:** inject `tests/`, run them; then `StopRuntimeSession`.

Table 12 Run settings for Terminal-Bench rows (`run.sh`)

`RB` × `NS` = `HARNESS_WORKERS`	4 × 8 = 32 fresh VMs per step
`EPISODE_MAX_TURNS` / `_TOKENS`	8 / 1024 per turn
`TB_EXEC_TIMEOUT`	180 s per bash command
`MAX_TOKENS_PER_GPU`	16384: one trajectory per micro-batch
`TB_PREOPEN_BATCHES`	2 batches booted ahead (slide 11)
`WARM_IDLE_TIMEOUT` × `PING_FRACTION`	900 × 0.5 = ping at 450 s (slide 14)

Source: `agentcore_slime_demo/terminalbench_pool.py` (`_run_episode`, `grade_episode`), `image_routing.py`; `demo_cpu_terminalbench/run.sh`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Only 1–2 warm VMs per Runtime: most concurrent opens pay a cold boot of 6–49 s

Figure 4 Open time vs image size (4 sessions at once per Runtime)

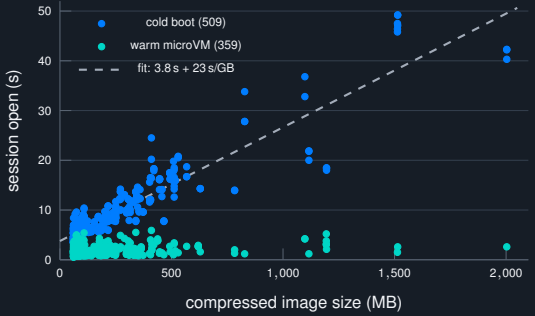


Table 13 Cold-start probe, all 218 Runtimes (2026-09-29)

Sessions	Warm	Open p50 / max
1st per Runtime	217 / 217	0.6 / 3.3 s
4 at once per Runtime	359 / 868	4.7 / 49 s
Cold opens by image size		
0–250 MB	n = 359	6.1 / 11 s
250–500 MB	107	12.1 / 25 s
500–1000 MB	26	16.7 / 34 s
1–2 GB	17	**40.3 / 49 s**

- **Bursty:** 4 at once → 509 of 868 cold; 160 of 217 Runtimes had exactly 1 warm. 8 GRPO siblings would fare worse (not probed).
- **Costly:** +32 % on a 21 s median episode.
- **No warm reuse:** a reset would leak OS state; **the session is the isolation boundary.**

Source: `terminalbench/probe_coldstart.py` → `tasks/coldstart.csv` (1,085 of 1,090 sessions ok; conc 16), us-west-2; smoke run 2026-09-23 (`episode_ms`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Pre-open replays slime's data order and boots the next 64 episodes' sessions ahead

`tb_lookahead_source.py` (simplified; a `--data-source-path` plugin)

```
def get_samples(self, n):
    groups = super().get_samples(n) # slime's own draw
    rb, ns = args.rollout_batch_size, args.n_samples_per_prompt
    future = self._peek(TB_PREOPEN_BATCHES * rb) # no state moved
    drawn = [arn_for(g[0])] * ns for g in groups
    upcoming = [arn_for(s)] * ns for s in future
    pool.reserve(drawn, upcoming) # boot ahead
    return groups
```

`TerminalBenchPool` (simplified)

```
def _refill(self): # on every draw and claim
    need = Counter((demand + future)[:P]) # P = 64
    for arn in need: # open or stop per task
        while len(ready[arn]) < need[arn]:
            ready[arn].append(open_session(arn))
def _claim(self, arn): # a worker starts an episode
    s = ready[arn].popleft(); self._refill(); return s
```

1. **No slime change:** `_peek` replays slime's seeded shuffle.
2. **Window P** = 2 batches × RB × NS = **64** booted sessions; wrong guesses are stopped.
3. **Each claim** starts one more boot.
4. **Failures retry** on a new id, never scored.

Rejected: reset a warm session (leaks) · pool per task (1,328 idle VMs).

Live microVMs $\leq$ W + P = 32 + 64. Quota: **5000** active sessions, 25 new/s [Q2] (slide 19).

Source: `agentcore_slime_demo/tb_lookahead_source.py`, `terminalbench_pool.py` (`reserve`, `_refill`, `_claim`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

With pre-open 374 of 382 episodes never waited for a boot; the price is idle VM time

Table 14 Warm-start probe: the same 24 episodes, batches of 6, cold vs pre-open (all rewards 1.0)

Backend	Mode	Wait p50	Episodes that waited	Idle held	Makespan
Own Runtime	without	774 ms	24 / 24	0 s	31.9 s
Own Runtime	**with**	**5 ms**	8 / 24 (first batch)	75 s	25.5 s (−20 %)
Code Interpreter	without	1715 ms	24 / 24	0 s	28.7 s
Code Interpreter	**with**	**0 ms**	6 / 24 (first batch)	70 s	24.3 s (−15 %)

Table 15 Full training smoke run with pre-open (382 episodes, P = 64)

Boot on the critical path (`open_wait_ms`)	**p50 0 · p95 0**; 8 of 382 waited: 4 at step 1 ($\leq$ 4.9 s), 4 at step 10 ($\approx$ 1 s)
Session open (`open_ms`)	p50 0.66 s · p95 2.5 s · 382 / 382 pre-opened · 0 retries
Price: pre-opened VM idle	p50 **7.3 min**, max 16.5 min, for a 21 s median episode

- **Sizing rule:** boot is hidden when $\mathbf{P \cdot E / W \geq B}$ (B boot, E episode, W workers).
- **P = 64 was generous:** P = 32 should halve idle time (not yet run).

Source: Warm-start probe 2026-09-25 (`terminalbench/warmstart/`, host-only, MockPolicy); smoke run 2026-09-23 (`_smoke_logs/lanes_tb_0923/terminalbench.demo.csv`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

An idle session is silently wiped after 15 min, and pre-opened sessions idle up to 16.5 min

What AgentCore does

- No invoke for **15 min** → session reclaimed [D2].
- **Silently:** the same id then gets a **fresh, empty sandbox**.

Why our sessions go idle

- **Pre-open:** a reclaimed session re-boots on use: **the boot is back**.
- **Weight sync:** a reclaim wipes the agent's work: **a wrong 0**.

Table 16 **Pre-open idle in the smoke run (382 sessions)**

Idle p50 / p90 / max	7.3 / 9.5 / 16.5 min
Idle past 450 s (ping point)	172
Idle past **900 s** (reclaim)	**17 (4.5 %)**

Table 17 **Probe: 60 s idle timeout, sessions idle 150 s**

Backend	Heartbeat	Alive	Resume
Own Runtime	on (14 pings)	**3 / 3**	104 ms, files intact
Own Runtime	off	0 / 3	same id, **empty sandbox**
Code Interpreter	on / off	0 / 3	ends at `sessionTimeoutSeconds`

Source: AgentCore docs [D2]; warm-start probe 2026-09-25 (`terminalbench/warmstart/`, Runtime `acs_warmstart_cpu`); smoke run 2026-09-23 (`preopen_idle_ms`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

One process-wide keeper pings any session idle past half the timeout: 348 pings, 0 failed

`session_keeper.py` (simplified)

```
class SessionKeeper: # one per rollout process
    def register(self, sid, pinger, idle_timeout): ...
    def touch(self, sid): # every real invoke
        self._sessions[sid].last = now()
    async def _run(self): # background loop
        while True:
            for s in self._sessions.values():
                if now() - s.last >= s.idle_timeout * 0.5:
                    try:
                        await s.pinger() # exec("true")
                        s.last = now()
                    except Exception: # retry next tick,
                        s.ping_failures += 1 # clock NOT reset
            await asyncio.sleep(tick)
```

`TerminalBenchPool._open_session`

```
async def _ping(s=sb): await s.exec("true", timeout=30)
keeper.register(sb.session_id, _ping, WARM_IDLE_TIMEOUT)
```

- **Outside the harness:** registered on open, removed on stop.
- **Cheap:** one `exec "true"`; busy sessions are never pinged.
- **A failed ping** retries next tick.

Table 18 **Smoke run (900 s timeout, pings at 450 s)**

Pings · sessions pinged	**348** · 214
of which idle only after the claim	42
Failed pings	0
Idle past 900 s, pinged in time	**17 / 17**

Source: `agentcore_slime_demo/session_keeper.py`, `terminalbench_pool.py` (`_open_session`); smoke run 2026-09-23 (`heartbeat_session`, `total_pings`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

The smoke run answers three questions: yes, yes, and not yet

Table 19 Smoke run 2026-09-23: Qwen3-4B, RB = 4 × NS = 8, 10 steps + 1 eval on 31 held-out tasks, ≈ 70 min

Question	Answer	Evidence	Slide
1. Does one-image-per-task RL run end to end?	Yes	382 episodes (train + eval) on 382 fresh microVMs, 71 tasks; **0** infra errors, retries, grading timeouts	below
2. Is the sandbox off the critical path?	Yes	boot wait p95 **0 ms** (pre-open); all 17 over-idle sessions pinged in time (heartbeat)	slides 12–14
3. Does the policy learn?	Not yet	4 of 320 train, 0 of 62 eval solved; 8 of 10 steps had zero gradient	slide 17

Table 20 Where an episode's time goes (p50)

Wait for a session (boot)	0 s
Agent loop in the microVM	9.0 s
Grading (inject tests, pytest)	9.7 s
Whole episode p50 (p90)	20.9 s (63.7 s)

- **Rollout-bound:** a step is 203 s p50; the trainer waits on the slowest episode, not on boots.
- **Grading is half the episode:** it installs `uv` + pytest each time.

Source: `_smoke_logs/lanes_tb_0923/terminalbench.{csv,demo.csv,eval.csv}` (`open_wait_ms`, `agent_ms`, `grade_ms`, `perf/step_time`); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

The 2026-09-30 re-run: 22 minutes end to end, 0 infra errors

Table 21 Smoke run 2026-09-30: same shape (RB = 4 × NS = 8, 10 steps + 1 eval on 31 held-out tasks), fully-async driver

Metric	2026-09-30	vs 2026-09-23 (slide 15)
Wall clock, launch to final eval	**22 min**	≈ 70 min: the fully-async driver overlaps rollouts with training
Episodes · infra errors	449 · **0**	382 · 0; 449 = 387 train (async over-draw) + 62 eval
Train episodes that waited for a boot	**0 of 385**	pre-open still hides every cold start; eval opens on demand (1–10 s)
Heartbeat pings needed	1	pre-open idle p50 50 s, max 194 s, all under the 450 s ping mark
Step time p50	40 s	203 s
Train solved · eval solved	22/387 (5.7 %) · **1/62**	4/320 · 0/62
GRPO groups with a gradient	4 of 49	2 of 40: reward stays the bottleneck (slide 17)

Table 22 Where a train episode's time goes (p50 / p90)

Wait for a session (boot)	0 s / 0 s
Agent loop in the microVM	12.1 s / 36.2 s
Grading (inject tests, pytest)	8.9 s / 20.6 s
Whole episode	24.9 s / 71.7 s

- **Concurrency:** 2 SGLang engines (4 GPUs), 32 harness workers, up to 64 pre-opened sessions; SGLang queued requests in 1 of 233 samples (max 15 running per engine).
- **Fixed in this run:** eval used to hang after its last episode (the pool was bound to the training loop); now one pool per event loop.

Source: `demo_cpu_terminalbench/logs/smoke_tbonly-agentcore-runtime_20260930_153115.{csv,demo.csv,eval.csv}`; checked 2026-09-30.

© 2026, Amazon Web Services, Inc. or its affiliates.

41 % of episodes pass a hidden test, but all-or-nothing reward left 38 of 40 GRPO groups with no gradient

Figure 5 Share of training episodes passing all vs $\geq$ 1 hidden test



- **4 solves** in 2 tasks; eval **0** of 62. No trend over 10 steps.

Table 23 GRPO groups (one task $\times$ 8 samples in a step)

Groups (40 tasks $\times$ 8)	40
Mixed under binary reward	**2**
Mixed under test-fraction reward	**17**
Steps with non-zero grad norm	2 of 10

Table 24 Why episodes stopped

Hit the 8-turn budget	140 (44 %)
Hit the 1024-token turn limit	98 (31 %)
Agent said it was done	82 (26 %)

An all-0 group has zero advantage; 74 % of episodes ran out of budget.

Source: `lanes_tb_0923/terminalbench.{demo,eval,}.csv` (`tests_passed`, `stop`, `train/grad_norm`; 320 training rows by step and task); checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

Next: fix the reward and the budget, then halve the idle cost

Table 25 Next steps, in order

	Change	Why (evidence)
1	Partial credit from `tests_passed` / `tests_total`	2 → 17 of 40 groups with a gradient (slide 17)
2	Larger turn and token budget (e.g. 16 turns, 2048 tokens)	74 % of episodes stopped on a budget
3	Easy-first curriculum over the 165 tasks	4B model solves 1 % (4 of 320); start where it sometimes succeeds
4	`TB_PREOPEN_BATCHES` = 1	pre-opened VMs idle 7.3 min p50; one batch should still hide the boot (slide 12)
5	Bake `uv` + pytest into each image	grading is 9.7 s of a 20.9 s episode, mostly the install (slide 15)

In plain English: the plumbing works and costs no waiting time; the task set is too hard for a 4B model with all-or-nothing reward, so the next run is about the reward, not the sandbox.

Source: Smoke run 2026-09-23, `code/demos/_smoke_logs/lanes_tb_0923`; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.

AgentCore quotas and docs back every limit used in this deck

Table 26 References

Ref	Source	Supports
[Q1]	AgentCore quotas: docs.aws.amazon.com/bedrock-agentcore/latest/devguide/bedrock-agentcore-limits.html	1000 Runtimes/account · image $\leq$ 2 GB · 15-min request timeout · 15-min idle timeout
[Q2]	`aws service-quotas list-service-quotas --service-code bedrock-agentcore` (us-west-2, 2026-09-23)	Active Session Workloads = 5000 (L-3E5722B2) · new sessions 25/s (L-8EE2AEA2) · image 2048 MB (L-0A9E32B3)
[D1]	AgentCore Runtime developer guide	microVMs are arm64 only
[D2]	docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-how-it-works.html	Session = dedicated microVM; 15-min idle, 8 h max lifetime

Source: AWS documentation and service-quotas CLI, us-west-2, 2026-09-23; checked 2026-09-29.

© 2026, Amazon Web Services, Inc. or its affiliates.